TASK 11 – Recommend and Defend the Final Design 4 Marks
Bloom's Level: K5 – Evaluate

Recommend the final set of data types, constants and preprocessor features for the co-operative.

Discuss:

• The types you recommend and why

• How a tariff revision will be carried out

• The limitations of your design

• Whether the design remains correct at district scale

Your recommendation must be based on evidence from your own experiments.
Alternative Concept and Complexity Analysis

If the tariff were read from a configuration file when the program starts, instead of being

compiled into the program, what would the solution be? Analyse the extra start-up time to read t tariff values and the space they occupy, against the cost of recompiling and redistributing the

program after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

1. Recommended Data Types and Constants

I recommend using int for ordinary unit counts, long for meter readings, and long long for district-level monetary totals in paise. Integer paise avoids floating-point rounding errors when storing money.

Tariff rates and fixed charges should be defined using named constants and the #define preprocessor directive. This improves readability and avoids repeating values throughout the program.

2. Tariff Revision

When the tariff changes, the named rate constants can be updated in the source code. The program must then be recompiled and the updated executable distributed. This makes tariff changes consistent throughout the program.

3. Limitations

The design depends on the integer sizes supported by the compiler. Meter rollover assumptions must match the actual meter. The program also needs input validation and tests for invalid readings. Tariff values should be verified before use.

4. Correctness at District Scale

The experiments compare totals for 4,000, 60,000 and 800,000 bills. The total for 800,000 bills at ₹162.50 each is ₹13,00,00,000, or 13,000,000,000 paise. A 32-bit signed int or long cannot hold this paise total, whereas a typical 64-bit long long can. Therefore, long long is recommended for district-level totals, provided multiplication and addition are checked for overflow.

5. Alternative: Read Tariffs from a Configuration File

The program could read the tariff rates and fixed charge from a configuration file when it starts. If there are t tariff values, reading them takes O(t) time and O(t) storage. After loading, the program can calculate bills without recompilation for each tariff revision.

This approach makes tariff updates easier, but the program must validate the file and handle missing or invalid values. Compiling constants into the program requires rebuilding and redistributing the executable after every revision. A configuration file avoids that repeated work, although it adds file-reading and validation overhead at startup.

Code :

```
#include <stdio.h>

#include <limits.h>

#define RATE1_PAISE 0LL

#define RATE2_PAISE 225LL

#define RATE3_PAISE 450LL

#define FIXED_CHARGE_PAISE 5000LL

long long calculate_bill(long long units)

{

  long long bill = FIXED_CHARGE_PAISE;

  if (units > 200)

    bill += (units - 200) * RATE3_PAISE;
```

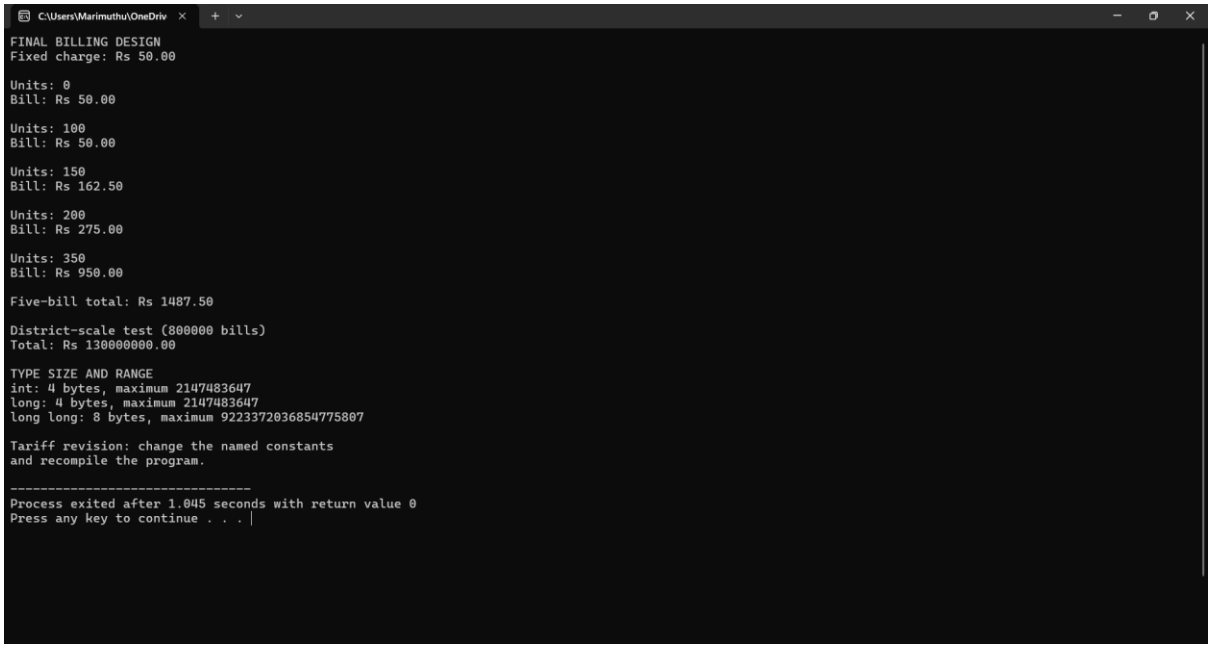
```
    if (units > 100) {
        long long slab2;
        if (units < 200)
            slab2 = units - 100;
        else
            slab2 = 100;
        bill += slab2 * RATE2_PAISE;
    }
    return bill;
}
int main(void)
{
    long long bill;
    long long district_total = 0;
    long long bills[] = {0, 100, 150, 200, 350};
    int i;
    printf("FINAL BILLING DESIGN\n");
    printf("Fixed charge: Rs 50.00\n");

    for (i = 0; i < 5; i++) {
        bill = calculate_bill(bills[i]);
        district_total += bill;
        printf("\nUnits: %lld\n", bills[i]);
        printf("Bill: Rs %lld.%02lld\n",
               bill / 100, bill % 100);
    }
    printf("\nFive-bill total: Rs %lld.%02lld\n",
           district_total / 100, district_total % 100);
```

```
    printf("\nDistrict-scale test (800000 bills)\n");
    district_total = 800000LL * calculate_bill(150);
    printf("Total: Rs %lld.%02lld\n",
        district_total / 100, district_total % 100);
    printf("\nTYPE SIZE AND RANGE\n");
    printf("int: %lu bytes, maximum %d\n",
        (unsigned long)sizeof(int), INT_MAX);
    printf("long: %lu bytes, maximum %ld\n",
        (unsigned long)sizeof(long), LONG_MAX);
    printf("long long: %lu bytes, maximum %lld\n",
        (unsigned long)sizeof(long long), LLONG_MAX);
    printf("\nTariff revision: change the named constants\n");
    printf("and recompile the program.\n");

    return 0;
}
```

Output :



Conclusion

Based on the experiments, I recommend named tariff constants, integer paise for individual bills, and long long for district-level totals. The design is suitable for district-scale use when range checks, input validation and correct meter-rollover handling are included.